Technical University of Moldova
Faculty of Computers, Informatics and Microelectronics
Department of Software Engineering and Automation

REPORT

Laboratory work No. 3

The Command Line: Shell, Pipes and Text Filters

Rotaru Anastasia, FAF-242

Course: **Operating Systems**

Checked by: Patricia Reițman, asist. univ.

Chişinău, 2026

Contents

1 Goal of the work **2**

2 Part 1. Move fast, find help **2**
2.1 Observations 2

3 Part 2. What the shell does to your line **3**
3.1 Predictions and outputs 3
3.2 Observations 3

4 Part 3. Redirection, chaining and jobs **4**
4.1 Observations 4

5 Part 4. Answer questions with one-liners **5**
5.1 Observations 5

6 Part 5. Your first script **5**
6.1 Observations 6

7 Conclusion **6**

1 Goal of the work

The primary purpose of this practical assignment was to develop advanced proficiency in navigating the Bash command-line interface, anticipating command-line expansions, managing stream redirections, and extracting meaningful information from raw datasets using text-filtering tools connected via pipelines. All experiments were conducted within the `UbuntuLab` virtual machine environment on `Laptop-Rotaru`, accurately capturing the input commands and corresponding system responses.

2 Part 1. Move fast, find help

```
nastea@UbuntuLab:/$ type cd; type ls; type echo; which ls
cd is a shell builtin
ls is aliased to ‘ls --color=auto’
echo is a shell builtin
/usr/bin/ls

nastea@UbuntuLab:/$ echo $PATH | tr ’:’ ’\n’
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin

nastea@UbuntuLab:/$ nosuchprog; echo "exit code: $?"
nosuchprog: command not found
exit code: 127
```

2.1 Observations

1. **Built-ins versus externals:** Instructions such as `cd` and `echo` are implemented directly within the core shell interpreter, whereas `ls` corresponds to an external executable binary. Specifically, `cd` must remain an internal built-in because a child process lacks the ability to alter the working directory of its parent process (the shell).

2. **PATH variable evaluation:** My active environment path string contains 9 distinct directory paths. The physical executable file for the `ls` utility is located at `/usr/bin/ls`.

3. **Exit codes:** Attempting to invoke a non-existent command instructs the system to return the standard error status code of `127`.

4. **Manual reference:** Reviewing the system manual page via `man ls` indicates that sorting directory listings by modification timestamp requires the `-t` flag.

3 Part 2. What the shell does to your line

3.1 Predictions and outputs

Command	My prediction	Real output	Explanation (if different)
`echo *.c`	a.c b.c	a.c b.c	Shell expands the wildcard to matching files.
`echo file?.txt`	file1.txt file2.txt	file1.txt file2.txt	`?` matches exactly one character.
`echo [a-b]*`	a.c b.c	a.c alfa beta.txt b.c	`*` matches the remainder of strings starting with a or b.
`echo *.xyz`	error / nothing	*.xyz	If no match exists, the literal string is passed.
`ls alfa beta.txt`	'alfa beta.txt'	cannot access 'alfa'...	Unescaped spaces act as argument separators.
`ls "alfa beta.txt"`	'alfa beta.txt'	'alfa beta.txt'	Quotes treat the string as a single argument.
`echo $HOME "$HOME"`	/home/nastea twice	/home/nastea /home/nastea	Double quotes still evaluate variables.
`echo '$HOME' \$HOME`	literal text	$HOME $HOME	Single quotes and backslashes escape variables.
`echo {1..5}; echo $((7*6))`	1 2 3 4 5 and 42	1 2 3 4 5 42	Brace expansion and arithmetic evaluation.

3.2 Observations

1. **Globbing mechanics:** Filename substitution is managed entirely by the shell interpreter prior to command execution. Patterns such as `*.c` are matched against actual files residing in the current directory and replaced with their names before being passed to `echo`.

2. **Correcting command invocation:** Unquoted spaces fragment input tokens into separate arguments (`alfa` and `beta.txt`). This can be resolved either by wrapping

the string in quotes (`ls "alfa beta.txt"`) or by escaping the whitespace character (`ls alfa\ beta.txt`).

3. **Quote evaluation behavior:** Double quotes permit environment variable expansion (such as `$HOME`), whereas single quotes suppress all forms of interpretation, treating expressions literally. Modern iterations of `ls` enclose filenames containing spaces within single quotes for visual clarity.

4 Part 3. Redirection, chaining and jobs

```
nastea@UbuntuLab:~/os-lab3$ ls /etc/nope > out.txt 2> err.txt
nastea@UbuntuLab:~/os-lab3$ wc -l out.txt; cat err.txt
0 out.txt
ls: cannot access '/etc/nope': No such file or directory

nastea@UbuntuLab:~/os-lab3$ wc -l < access.log
500

nastea@UbuntuLab:~/os-lab3$ ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2

nastea@UbuntuLab:~/os-lab3$ ls /etc/nope > all.txt 2>&1
nastea@UbuntuLab:~/os-lab3$ ls /etc/nope 2>&1 > all.txt
ls: cannot access '/etc/nope': No such file or directory
```

4.1 Observations

1. **Stream separation:** Diagnostic error messages were routed to `err.txt` via standard error redirection (`2>`), while the standard output file `out.txt` remained empty because the command encountered a failure.

2. **Redirection sequence logic:** The interpreter evaluates redirection rules sequentially from left to right. Using `> all.txt 2>&1` binds stdout to the file first and then points stderr to whatever destination stdout currently occupies. Reversing this order as `2>&1 > all.txt` maps stderr to the terminal screen initially before stdout is redirected to the file, leaving error logs visible on-screen.

3. **Feeding standard input:** Executing `wc -l < access.log` supplies data through standard input, meaning `wc` processes the stream without retaining knowledge of the filename. Passing the file directly as an argument (`wc -l access.log`) provides the filename to `wc`, allowing it to be output alongside the count.

4. **Logical chaining operators:**

   - The semicolon `;` runs successive commands unconditionally, regardless of preceding success or failure.
   - The operator `&&` (logical AND) triggers the subsequent command *only* if the previous operation exits successfully (status code 0).
   - The operator `||` (logical OR) triggers the subsequent command *only* if the previous operation encounters an error (non-zero exit code).

5 Part 4. Answer questions with one-liners

```
nastea@UbuntuLab:~/os-lab3$ bash oneliners.sh
100
    125 /page0
    125 /page1
    125 /page2
    125 /page3
    125 /page3
    125 /page2
    125 /page1
    125 /page0
     15 user5
Total requests by user3: 72 | Failed: 15
10:10 user0
10:15 user5
10:20 user3
    40 /usr/sbin/nologin
     6 /bin/false
     2 /bin/bash
     1 /bin/sync
Bonus - Failed at exact 10 minutes:
50
```

5.1 Observations

1. Why does `ls -l /etc | wc -l` return one more line than `ls /etc | wc -l`?
The long-listing format (`-l`) automatically prefixes output with an initial summary header row (e.g., `total 128`) prior to presenting individual file entries. Standard unformatted `ls` omits this summary line entirely, which explains why the total count generated by `-l` is higher by one unit.

6 Part 5. Your first script

```
nastea@UbuntuLab:~/os-lab3$ ./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures

nastea@UbuntuLab:~/os-lab3$ ./report.sh nope.log; echo "exit code: $?"
no such file: nope.log
exit code: 1

nastea@UbuntuLab:~/os-lab3$ cat ~/os-lab3/report.log
500 requests, 100 failed
user0: 14 failures
...
```

6.1 Observations

1. **Shebang directive usage:** The leading interpreter declaration `#!/bin/bash` instructs the operating system on which interpreter to use for script parsing. Attempting to run the file prior to granting execution permissions via `chmod +x` results in a standard "Permission denied" error.

2. **Local path specification:** Invoking the script must be performed explicitly via `./report.sh` to instruct the shell to check the current working directory (`.`). Omitting this prefix forces the shell to scan strictly through directories defined in `$PATH`, triggering a command-not-found warning.

3. **Error management:** Diagnostic notifications are directed to standard error using `>&2` rather than contaminating standard output, terminating with `exit 1` to signal failure status to parent processes and the operating system.

7 Conclusion

Throughout this laboratory assignment, I explored the practical functionalities of the Bash command-line interface on Linux. I examined how command lines undergo preprocessing via globbing and expansion, managed output and error streams through flexible redirection, and linked processes using logical operators such as `;`, `&&`, and `||`. By constructing multi-stage pipelines with text-processing filters like `grep`, `cut`, `sort`, and `uniq`, I efficiently extracted and analyzed statistical data from log files. Finally, packaging these solutions into a parameterized executable script reinforced core principles of shell scripting, exit code handling, and task automation.